UniCaronas

Sistema de Caronas Universitárias

Resumo da Reunião de Planejamento — Sprint 4

Engenharia de Software — 3ESCN

Período da Sprint: 16/04/2026 a 22/04/2026 (1 semana — Sprint final)

Função	Nome
Product Owner — define e esclarece os requisitos	Rafael Machado
Scrum Master — facilita a cerimônia	Ariane da Silva Archanjo
Dev Team — estima, planeja e se compromete	Pedro Kafka, Matheus Sizanoski, Rafael Machado, Ariane da Silva Archanjo

---

1. Como a cerimônia aconteceu

A reunião de Sprint Planning foi realizada em 16/04/2026 de forma presencial. Por ser a Sprint final do projeto, o foco foi diferente das anteriores: além de planejar as últimas funcionalidades, a equipe reservou tempo para discutir polimento geral, cobertura de testes e preparação para a apresentação final. O Product Owner apresentou o backlog restante e a equipe priorizou conjuntamente o que agrega mais valor dentro de uma semana.

Campo	Detalhe
Data	09/04/2026 (quinta-feira)
Formato	Presencial
Duração	Aproximadamente 1h30
Quem conduziu	Ariane da Silva Archanjo (Scrum Master)
Quem definiu as tarefas	Rafael Machado (Product Owner)
Início da Sprint	09/04/2026
Fim da Sprint	15/04/2026 — apresentação final
Caráter	Sprint de encerramento — foco em qualidade, avaliações e polimento

---

2. O que será entregue ao final da Sprint

Por ser a Sprint final, a entrega engloba tanto as últimas funcionalidades quanto a estabilização completa do sistema para apresentação. Ao final desta Sprint, o UniCaronas deve estar pronto para demonstração completa: cadastro, login, criação de carona, busca, solicitação, aceite, chat, pagamento, avaliação e histórico — tudo funcionando de ponta a ponta.

A entrega está completa quando:

- Sistema de avaliações funcional — motorista e passageiro se avaliam após a carona
- Histórico de caronas concluídas visível na tela de perfil com papel (motorista / passageiro)
- Tela de perfil exibindo avaliações recebidas com estrelas, comentário e rota da carona
- Caronas recorrentes (semanais) criando automaticamente as próximas 3 semanas
- README atualizado com instruções completas de instalação e uso
- Banco de dados com dados de teste realistas para demonstração
- Fluxo completo testado e sem erros conhecidos — pronto para apresentação

---

3. Quais histórias de usuário serão trabalhadas

O Product Owner apresentou as histórias finais do backlog. A Sprint 4 tem caráter de encerramento: além de novas funcionalidades, inclui estabilização, polimento de interface e preparação para a apresentação final.

ID	Funcionalidade	Pontos estimados	Prioridade	Sprint
US10	Avaliar motorista ou passageiro após carona	5 pontos	Alta	4
US11	Histórico de caronas no perfil	3 pontos	Alta	4
US12	Caronas recorrentes (semanais)	3 pontos	Média	4
US13	Polimento, testes e documentação final	2 pontos	Alta	4
—	Total	13 pontos	—	—

---

4. Por quem será feito

Membro	Responsabilidade principal
Rafael Machado	Product Owner: refinamento final, desenvolvimento do backend de avaliações e endpoint de histórico, seed de dados para apresentação
Ariane da Silva Archanjo	Scrum Master: facilitação, desenvolvimento da seção de avaliações na tela de perfil e renderização de estrelas
Pedro Kafka	Desenvolvimento: caronas recorrentes no controller, testes manuais do fluxo completo, polimento do frontend
Matheus Sizanoski	Desenvolvimento: histórico de caronas, atualização do README, preparação do ambiente de demonstração

---

5. Como cada funcionalidade será feita

US10 — Avaliar motorista ou passageiro após carona (5 pontos)

Como usuário que participou de uma carona concluída, quero avaliar o outro participante com nota de 1 a 5 e comentário opcional, para que a comunidade conheça a reputação de cada membro.

Como será feito:

- A tabela avaliacoes e o trigger trigger_avaliacao_media já existem no banco — sem migração
- O avaliacoesController.js já tem os endpoints POST /api/avaliacoes e GET /api/avaliacoes/:uid implementados
- Frontend: após carona concluída, botão 'Avaliar' aparece na tela de detalhes para ambos os participantes
- Modal de avaliação com seletor de estrelas (1–5) via radio buttons estilizados em CSS e campo de comentário
- Validar que avaliador e avaliado são participantes da mesma carona no backend
- Impedir auto-avaliação e avaliação duplicada (constraint UNIQUE no banco retorna 409)
- Trigger atualiza avaliacao_media e total_avaliacoes em usuarios automaticamente
- Avaliação média atualizada visível no perfil e nos cards de carona em tempo real

Para considerar pronto:

Critério de Aceitação	Status
Botão 'Avaliar' aparece após carona concluída para ambos os participantes	■ Planejado
Modal com seletor de estrelas (1–5) e campo de comentário opcional	■ Planejado
Avaliação duplicada retorna erro 409 com mensagem clara	■ Planejado
Média do usuário atualizada automaticamente após avaliação	■ Planejado
Avaliação visível na tela de perfil do avaliado	■ Planejado

US11 — Histórico de caronas no perfil (3 pontos)

Como usuário, quero visualizar no meu perfil o histórico de caronas concluídas, identificando se participei como motorista ou passageiro e o valor pago ou recebido.

Como será feito:

- Endpoint GET /api/caronas/historico/:usuario_id já implementado — retorna caronas concluídas com papel (motorista/passageiro)
- Frontend: seção 'Histórico de Caronas' na tela perfil.html abaixo das estatísticas
- Cada item exibe: origem → destino, data/hora formatada, papel (badge colorido) e valor
- Badge roxo para motorista, verde para passageiro — mesma paleta do design system
- Ordenado por data decrescente — mais recentes no topo
- Estado vazio com mensagem quando não há caronas concluídas

Para considerar pronto:

Critério de Aceitação	Status
Seção 'Histórico de Caronas' visível na tela de perfil	■ Planejado
Cada carona exibe rota, data, papel e valor	■ Planejado
Badge diferencia papel de motorista e passageiro visualmente	■ Planejado
Estado vazio exibido quando não há histórico	■ Planejado

US12 — Caronas recorrentes / semanais (3 pontos)

Como motorista que faz o mesmo trajeto toda semana, quero publicar uma carona como recorrente e ter as próximas semanas criadas automaticamente.

Como será feito:

- A lógica de caronas recorrentes já está parcialmente implementada no caronasController.js — verificar e finalizar
- Toggle 'Carona semanal' na tela criar-carona.html com switch visual já presente na interface
- Quando recorrente = true, o backend cria a carona principal + 3 cópias nas semanas seguintes
- Cada cópia herda origem, destino, vagas, valor e observações — apenas a data avança 7 dias
- Badge 'Semanal' exibido nos cards de busca e na tela de detalhes para caronas recorrentes
- Campo recorrente adicionado na tabela caronas se não existir (verificar schema)

Para considerar pronto:

Critério de Aceitação	Status
Toggle 'Carona semanal' funcional na tela de criação	■ Planejado
Backend cria automaticamente as próximas 3 semanas ao ativar toggle	■ Planejado
Badge 'Semanal' exibido nos cards de caronas recorrentes	■ Planejado
Cada cópia semanal é independente — vagas e status gerenciados separadamente	■ Planejado

US13 — Polimento, testes e documentação final (2 pontos)

Como equipe, queremos entregar o sistema estável, documentado e com dados de teste realistas para a apresentação final.

Como será feito:

- README.md atualizado com todas as rotas da API, instruções de instalação passo a passo e variáveis de ambiente
- Arquivo data.sql revisado com dados de teste realistas: usuários, caronas, solicitações, mensagens e avaliações prontos para demonstração
- Testes manuais do fluxo completo: cadastro → login → criar carona → buscar → solicitar → aceitar → chat → pagar → concluir → avaliar → ver histórico
- Correção de quaisquer bugs encontrados nos testes
- Revisão final de responsividade em dispositivos móveis e desktop

- Verificação de consistência visual em todas as telas — fontes, cores e espaçamentos

Para considerar pronto:

Critério de Aceitação	Status
README com instruções completas e rotas da API documentadas	■ Planejado
data.sql com dados realistas suficientes para demonstração completa	■ Planejado
Fluxo completo testado sem erros conhecidos	■ Planejado
Interface consistente e responsiva em mobile e desktop	■ Planejado

---

6. Tecnologias

Tecnologia	Para que será usada
HTML, CSS e JavaScript	Modal de avaliação com estrelas, seção de histórico no perfil, toggle de carona semanal
Node.js + Express	Integração dos endpoints de avaliações e histórico já existentes, ajuste do controller de caronas recorrentes
PostgreSQL	Tabela avaliacoes e trigger trigger_avaliacao_media — já existentes; seed de dados atualizado
JWT (token de acesso)	Todas as rotas protegidas — sem alterações
GitHub	Versionamento — entrega final na branch principal

---

7. Quando será testado

Por ser a Sprint final, os testes têm peso maior. Além dos testes por funcionalidade, a equipe realiza um teste de fluxo completo no penúltimo dia — simulando a demonstração real para o professor.

Data	O que é testado	Como é testado
09/04	US10 — Avaliações	Dois usuários se avaliam após carona concluída; verificar média atualizada no perfil e bloqueio de duplicata
10/04	US11 — Histórico de caronas	Usuário com caronas concluídas acessa perfil e verifica histórico com papel e valor corretos
10/04	US12 — Caronas recorrentes	Criar carona com toggle semanal ativo; verificar se 4 caronas são criadas no banco nas datas corretas
11/04	Fluxo completo	Simulação da apresentação: fluxo inteiro do zero com dados limpos. Product Owner valida cada etapa

Data	O que é testado	Como é testado
12/04	Correções finais	Resolver qualquer bug encontrado no dia anterior; revisão de README e dados de seed

---

8. Quando vai para produção

Nesta Sprint final, 'ir para produção' significa ter o código completo, documentado e funcionando para a apresentação final do projeto.

Data	O que acontece
13/04 (Seg)	Código finalizado e salvo na branch principal do GitHub — versão estável, documentada e sem erros conhecidos
13/04 (Ter)	Apresentação final do sistema completo — demonstração do fluxo end-to-end para a turma e para o professor

---

9. Visão Geral do Projeto ao Final da Sprint 4

Ao término desta Sprint, o UniCaronas estará com todas as funcionalidades principais implementadas. O quadro abaixo resume o backlog completo do projeto:

ID	Funcionalidade	Story Points	Sprint	Status
US01	Criar conta no sistema	5 SP	Sprint 1	✓✓ Entregue
US02	Login / Autenticação JWT	3 SP	Sprint 1	✓✓ Entregue
US03	Ver e editar perfil	3 SP	Sprint 1	✓✓ Entregue
US04	Criar carona	5 SP	Sprint 2	✓✓ Entregue
US05	Sugestão de valor por distância	3 SP	Sprint 2	✓✓ Entregue
US06	Buscar caronas com filtros	5 SP	Sprint 2	✓✓ Entregue
US07	Solicitar vaga em carona	5 SP	Sprint 2	✓✓ Entregue
US08	Chat entre usuários	5 SP	Sprint 3	✓✓ Entregue
US08b	Aceitar/Recusar pela interface	3 SP	Sprint 3	✓✓ Entregue
US09	Realizar pagamento (simulado)	5 SP	Sprint 3	✓✓ Entregue
US10	Avaliar motorista / passageiro	5 SP	Sprint 4	■ Planejado

ID	Funcionalidade	Story Points	Sprint	Status
US11	Histórico de caronas no perfil	3 SP	Sprint 4	■ Planejado
US12	Caronas recorrentes / semanais	3 SP	Sprint 4	■ Planejado
US13	Polimento, testes e docs	2 SP	Sprint 4	■ Planejado
—	Total do projeto	55 SP	4 sprints	—